- Introduction (5%): Please provide a high-level introduction to your report. You can mention the most important findings and the overall organization of this report.

這次的 lab 要實作 A2C 和 PPO 兩種演算法用於 Gymnasium 的 Pendulum 環境和 MuJuCo 的 Walker2d 環境。A2C 和 PPO 主要的差別在於 PPO 加上了新舊策略比例的控制、GAE 和更新網路的頻率，因此 PPO 能收斂得更快、更穩定。另外，也有對一些參數和設定進行實驗。報告中我會先說明我實作的方法，再展示結果。

- Your implementation (20%): Please briefly explain both the concepts and your implementation details for Tasks 1–3. For each item below, you should first describe how the method works (e.g., how to compute TD error) and then explain how you implemented it in your code. Specifically, please describe:

  - How do you obtain the stochastic policy gradient and the TD error for A2C?

stochastic policy gradient 要計算某個動作在整體分布中所佔的機率，

$$\nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

根據公式，要讓此梯度上升，而在 pytorch 中對應的是讓它的相反數遞減。

```
policy_loss = -(log_prob.view(-1) * advantage.view(-1)).mean() - self.entropy_weight * entropy
```

程式碼中先把梯度乘以 TD error，再額外減去用來增加探索的 entropy。

當中的 TD error 是用來衡量某個 action 下的 Q value 相較於平均值好多少

$$r_{t+1} + \gamma V_k(s_{t+1}) - V_k(s_t) =: \delta_t \text{ is called the TD error}$$

依照老師上課投影片的公式來計算，TD error 要把做完某個 action 得到的 reward 加上 discount factor(降低未來價值的權重)乘以未來的價值，減去當前的價值。如果 terminate 了，則不會有未來的價值，只使用 reward 來計算。

```
if not terminated:
    Q_t = reward + self.gamma * self.critic(next_state).view(-1)
else:
    Q_t = reward
```

```
V_t = self.critic(state).view(-1)
```

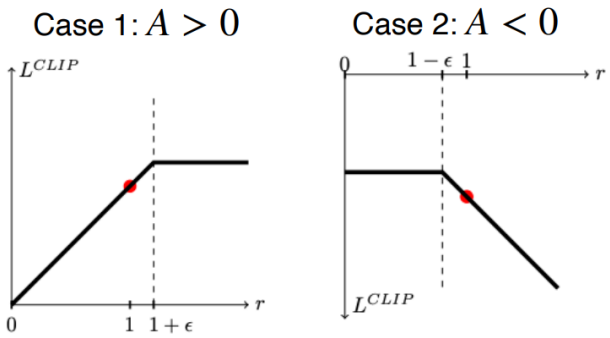
```
advantage = (Q_t - V_t).detach()
```

- How do you implement the clipped objective in PPO?

Clip 的目的是為了避免網路更新時的動作分布機率變化太大，所以限制了新策略和舊策略之間相對的比例。

Clip the ratio $\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}$

$$\max_\theta \mathbb{E}_{s\sim d_\mu^{\pi_{\theta_k}}, a\sim\pi_{\theta_k}(\cdot|s)}\left[\min\left\{\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}A^{\theta_k}(s,a), \text{clip}\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}, 1-\varepsilon, 1+\varepsilon\right)A^{\theta_k}(s,a)\right\}\right]$$



根據公式，如果 A 大於 0 時，會把 loss 的上限裁剪掉，避免模型過於樂觀。而如果 A 小於 0 時，代表動作表現不如預期，此時模型會調低該動作的機率，為了避免單次更新幅度太大，clip 會限制範圍。

```
surr_loss = adv * ratio
clipped_loss = adv * torch.clamp(ratio, 1.0 - self.epsilon, 1.0 + self.epsilon)
entropy = dist.entropy().mean()
actor_loss = -torch.min(surr_loss, clipped_loss).mean() - entropy * self.entropy_weight
```

程式碼中依據定義照著做，同時加上了 entropy 用來增加探索。

- How do you obtain the estimator of GAE?

GAE 是為了在蒙地卡羅與 TD error 之間取得平衡，平衡其中的 bias 和 variance。

GAE Estimator: $\hat{A}_t^{GAE(\gamma,\lambda)} = (1-\lambda)\left(\hat{A}_t^{(1)} + \lambda\hat{A}_t^{(2)} + \lambda^2\hat{A}_t^{(3)} + \cdots\right) = \sum_{\ell=0}^{\infty}(\gamma\lambda)^{\ell}\delta_{t+\ell}$

根據公式，GAE 在計算時是把不同次的 delta 累加起來，再乘以一個倍數，呈現指數遞減，而由這個公式可以進一步改成遞迴式，先計算 reward 加上 value 的差距，再加上後一個動作的 gae 的一部份，不斷循環。

```
for i in reversed(range(len(rewards))):
    delta = rewards[i] + masks[i] * gamma * values[i + 1] - values[i]
    gae = (masks[i] * gamma * tau) * gae + delta
    gae_returns.append(gae + values[i])
```

以上程式碼在 compute_gae 函數中。

- How do you collect samples from the environment?

在 A2C 中，每跟環境互動一次，就使用互動的 transition 更新網路，而 PPO 則會跟環境互動達到 rollout_len 次，在這期間的所有互動會先存起來，然後再從中用 mini batch 的方式抽樣來更新網路。

A2C 把狀態和機率存到 transition：

```
log_prob = dist.log_prob(selected_action).sum(dim=-1)
self.transition = [state, log_prob]
```

```
self.transition.extend([next_state, reward, done, terminated, truncated])
```

PPO 儲存和環境互動的資料：

```
self.states.append(state)
self.actions.append(selected_action)
self.values.append(value)
self.log_probs.append(dist.log_prob(selected_action))
```

```
self.rewards.append(torch.FloatTensor(reward).to(self.device))
self.masks.append(torch.FloatTensor(1 - done).to(self.device))
```

A2C 直接更新網路(放在 for 迴圈內，邊蒐集邊更新)：

```
actor_loss, critic_loss, mean_entropy = self.update_model(next_state)
```

PPO 會等蒐集夠多才更新網路(放在 for 迴圈外，蒐集完才更新)：

```
for _ in range(self.rollout_len):
```

```
actor_loss, critic_loss, mean_entropy = self.update_model(next_state)
```

- How do you enforce exploration (despite that both A2C and PPO are on-policy RL methods)?

不管是 A2C 還是 PPO 都使用了高斯分布，從中隨機抽樣，增加探索度。

```
x = self.network(state)
mu = self.mu(x)
std = self.std(x)
std = F.softplus(std) + 1e-4
dist = Normal(mu, std)
action = dist.sample()
```

```
x = self.network(state)
mu = torch.tanh(self.mu(x)) * 2.0
std = self.std(x)
std = torch.exp(torch.clamp(std, self.log_std_min, self.log_std_max))
dist = Normal(mu, std)
action = dist.sample()
```

```
x = self.network(state)
mu = torch.tanh(self.mu(x))
std = self.std(x)
std = torch.exp(torch.clamp(std, self.log_std_min, self.log_std_max))
dist = Normal(mu, std)
action = dist.sample()
```

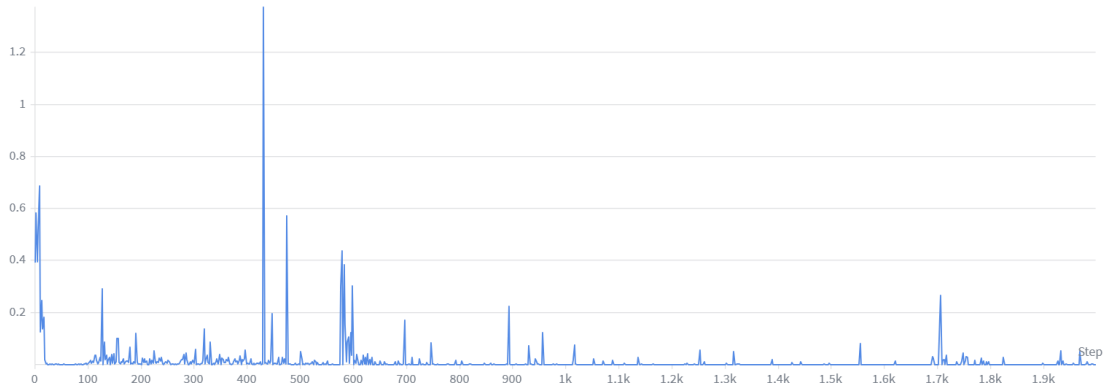
另外，也都有加上 entropy，透過在 loss 函數中加入 entropy，可以最大化動作的隨機性，增加探索度。

```
entropy = dist.entropy().mean()
policy_loss = -(log_prob.view(-1) * advantage.view(-1)).mean() - self.entropy_weight * entropy
```

```
entropy = dist.entropy().mean()
actor_loss = -torch.min(surr_loss, clipped_loss).mean() - entropy * self.entropy_weight
```

- Explain how you use Weight & Bias to track model performance and the loss values (including actor loss, critic loss, and the entropy).

在 Weight & Bias 中可以直接觀看 evaluation score 來判斷模型大致的表現，每隔固定步數就會評估一次，評估模型的分數是否有達到作業的標準。而觀看 artor loss 和 critic loss 則可以看出模型是否有收斂，例如下圖就能很明顯看出可能訓練失敗了，因為 loss 會有突然的暴增。Entropy 則是可以觀察模型是否有在探索，避免策略分布過窄。



在 Weight & Bias 紀錄 eval_score, loss, entropy 的程式碼：

```python
if step_count % eval_interval == 0:
    eval_score = self.evaluate()
    wandb.log({
        "env_step": step_count,
        "eval_score": eval_score
    })
```

```python
wandb.log({
    "env_step": step_count,
    "actor loss": actor_loss,
    "critic loss": critic_loss,
    "entropy": entropy
    })
```

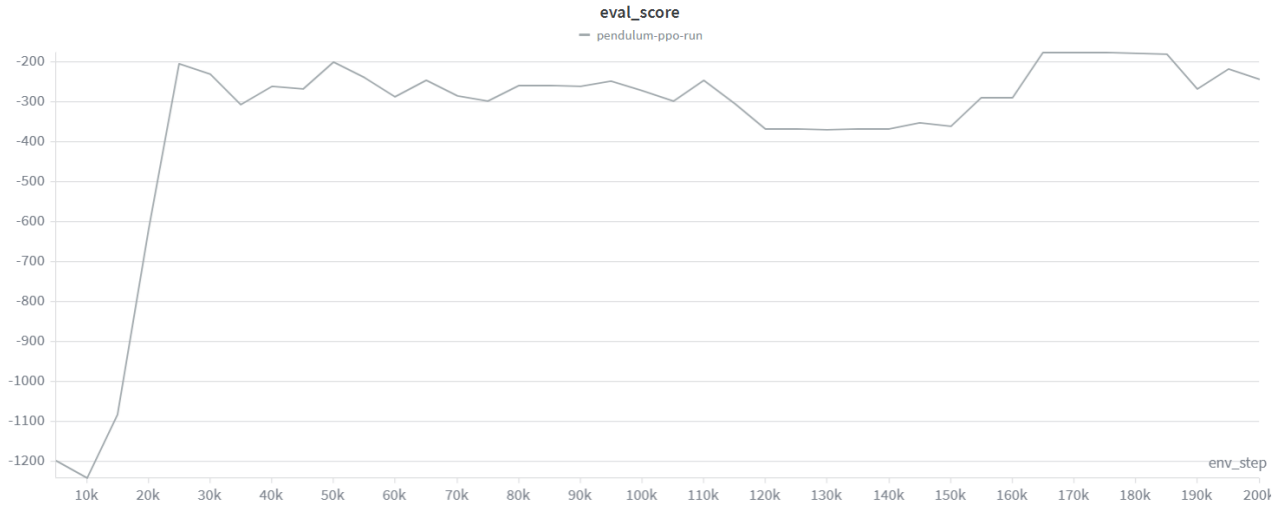
- Analysis and discussions (25%)

– Plot the training curves (evaluation score versus environment steps) for Task 1, Task 2, and Task 3 separately, please make sure the x-axis represent environment step (3+3+4=10%).

Task 1:



Task 2:



Task 3:



– Paste the screenshots of your reproducible evaluation results for 20 episodes and seed for Task1, Task2 and Task3, please make sure the environment step is included in the screenshot.

Task 1:

```
Environment steps: 190000, seed: 700, eval reward: -264.20
Environment steps: 190000, seed: 701, eval reward: -378.63
Environment steps: 190000, seed: 702, eval reward: -260.68
Environment steps: 190000, seed: 703, eval reward: -2.06
Environment steps: 190000, seed: 704, eval reward: -137.69
Environment steps: 190000, seed: 705, eval reward: -4.48
Environment steps: 190000, seed: 706, eval reward: -413.44
Environment steps: 190000, seed: 707, eval reward: -129.59
Environment steps: 190000, seed: 708, eval reward: -2.32
Environment steps: 190000, seed: 709, eval reward: -129.28
Environment steps: 190000, seed: 710, eval reward: -261.97
Environment steps: 190000, seed: 711, eval reward: -1.76
Environment steps: 190000, seed: 712, eval reward: -133.64
Environment steps: 190000, seed: 713, eval reward: -134.15
Environment steps: 190000, seed: 714, eval reward: -2.56
Environment steps: 190000, seed: 715, eval reward: -263.27
Environment steps: 190000, seed: 716, eval reward: -115.43
Environment steps: 190000, seed: 717, eval reward: -2.90
Environment steps: 190000, seed: 718, eval reward: -122.16
Environment steps: 190000, seed: 719, eval reward: -129.25
Average reward: -144.47
```

Task 2:

```
Environment steps: 30000, seed: 700, eval reward: -126.39
Environment steps: 30000, seed: 701, eval reward: -243.94
Environment steps: 30000, seed: 702, eval reward: -261.89
Environment steps: 30000, seed: 703, eval reward: -0.57
Environment steps: 30000, seed: 704, eval reward: -128.67
Environment steps: 30000, seed: 705, eval reward: -129.46
Environment steps: 30000, seed: 706, eval reward: -435.05
Environment steps: 30000, seed: 707, eval reward: -129.69
Environment steps: 30000, seed: 708, eval reward: -0.79
Environment steps: 30000, seed: 709, eval reward: -129.28
Environment steps: 30000, seed: 710, eval reward: -125.04
Environment steps: 30000, seed: 711, eval reward: -0.23
Environment steps: 30000, seed: 712, eval reward: -124.47
Environment steps: 30000, seed: 713, eval reward: -126.30
Environment steps: 30000, seed: 714, eval reward: -1.08
Environment steps: 30000, seed: 715, eval reward: -124.82
Environment steps: 30000, seed: 716, eval reward: -245.91
Environment steps: 30000, seed: 717, eval reward: -1.66
Environment steps: 30000, seed: 718, eval reward: -255.12
Environment steps: 30000, seed: 719, eval reward: -130.54
Average reward: -136.05
```

Task 3 (1M):

```
Environment steps: 1000000, seed: 77, eval reward: 3016.51
Environment steps: 1000000, seed: 78, eval reward: 3104.90
Environment steps: 1000000, seed: 79, eval reward: 2969.24
Environment steps: 1000000, seed: 80, eval reward: 2768.18
Environment steps: 1000000, seed: 81, eval reward: 789.50
Environment steps: 1000000, seed: 82, eval reward: 3116.92
Environment steps: 1000000, seed: 83, eval reward: 3029.80
Environment steps: 1000000, seed: 84, eval reward: 3025.15
Environment steps: 1000000, seed: 85, eval reward: 3011.73
Environment steps: 1000000, seed: 86, eval reward: 1248.51
Environment steps: 1000000, seed: 87, eval reward: 2753.92
Environment steps: 1000000, seed: 88, eval reward: 3105.96
Environment steps: 1000000, seed: 89, eval reward: 3135.04
Environment steps: 1000000, seed: 90, eval reward: 3141.76
Environment steps: 1000000, seed: 91, eval reward: 2994.34
Environment steps: 1000000, seed: 92, eval reward: 2922.75
Environment steps: 1000000, seed: 93, eval reward: 2764.46
Environment steps: 1000000, seed: 94, eval reward: 609.05
Environment steps: 1000000, seed: 95, eval reward: 3093.49
Environment steps: 1000000, seed: 96, eval reward: 3168.59
Average reward: 2688.49
```

Task 3 (1.5M):

```
Environment steps: 1500000, seed: 77, eval reward: 3564.05
Environment steps: 1500000, seed: 78, eval reward: 3612.60
Environment steps: 1500000, seed: 79, eval reward: 3729.79
Environment steps: 1500000, seed: 80, eval reward: 551.54
Environment steps: 1500000, seed: 81, eval reward: 3802.80
Environment steps: 1500000, seed: 82, eval reward: 3690.72
Environment steps: 1500000, seed: 83, eval reward: 3843.90
Environment steps: 1500000, seed: 84, eval reward: 2010.59
Environment steps: 1500000, seed: 85, eval reward: 2053.36
Environment steps: 1500000, seed: 86, eval reward: 3983.02
Environment steps: 1500000, seed: 87, eval reward: 3771.46
Environment steps: 1500000, seed: 88, eval reward: 1833.76
Environment steps: 1500000, seed: 89, eval reward: 2669.53
Environment steps: 1500000, seed: 90, eval reward: 3890.26
Environment steps: 1500000, seed: 91, eval reward: 916.91
Environment steps: 1500000, seed: 92, eval reward: 1217.24
Environment steps: 1500000, seed: 93, eval reward: 2959.39
Environment steps: 1500000, seed: 94, eval reward: 1678.61
Environment steps: 1500000, seed: 95, eval reward: 1283.65
Environment steps: 1500000, seed: 96, eval reward: 3629.98
Average reward: 2734.66
```

Task 3 (2M):

```
Environment steps: 2000000, seed: 77, eval reward: 505.71
Environment steps: 2000000, seed: 78, eval reward: 437.43
Environment steps: 2000000, seed: 79, eval reward: 1402.38
Environment steps: 2000000, seed: 80, eval reward: 1230.51
Environment steps: 2000000, seed: 81, eval reward: 1758.48
Environment steps: 2000000, seed: 82, eval reward: 3071.26
Environment steps: 2000000, seed: 83, eval reward: 437.76
Environment steps: 2000000, seed: 84, eval reward: 499.90
Environment steps: 2000000, seed: 85, eval reward: 1914.76
Environment steps: 2000000, seed: 86, eval reward: 3162.13
Environment steps: 2000000, seed: 87, eval reward: 3501.29
Environment steps: 2000000, seed: 88, eval reward: 609.59
Environment steps: 2000000, seed: 89, eval reward: 2358.13
Environment steps: 2000000, seed: 90, eval reward: 1041.34
Environment steps: 2000000, seed: 91, eval reward: 3666.01
Environment steps: 2000000, seed: 92, eval reward: 1447.24
Environment steps: 2000000, seed: 93, eval reward: 420.24
Environment steps: 2000000, seed: 94, eval reward: 1255.26
Environment steps: 2000000, seed: 95, eval reward: 1527.25
Environment steps: 2000000, seed: 96, eval reward: 1393.77
Average reward: 1582.02
```

Task 3 (2.5M):

```
Environment steps: 2500000, seed: 77, eval reward: 4069.77
Environment steps: 2500000, seed: 78, eval reward: 3231.53
Environment steps: 2500000, seed: 79, eval reward: 4114.65
Environment steps: 2500000, seed: 80, eval reward: 794.78
Environment steps: 2500000, seed: 81, eval reward: 2715.70
Environment steps: 2500000, seed: 82, eval reward: 3674.17
Environment steps: 2500000, seed: 83, eval reward: 1830.99
Environment steps: 2500000, seed: 84, eval reward: 787.04
Environment steps: 2500000, seed: 85, eval reward: 4025.53
Environment steps: 2500000, seed: 86, eval reward: 2321.19
Environment steps: 2500000, seed: 87, eval reward: 1455.54
Environment steps: 2500000, seed: 88, eval reward: 2227.94
Environment steps: 2500000, seed: 89, eval reward: 2410.46
Environment steps: 2500000, seed: 90, eval reward: 1251.08
Environment steps: 2500000, seed: 91, eval reward: 3128.32
Environment steps: 2500000, seed: 92, eval reward: 3956.22
Environment steps: 2500000, seed: 93, eval reward: 3763.10
Environment steps: 2500000, seed: 94, eval reward: 3022.36
Environment steps: 2500000, seed: 95, eval reward: 3757.39
Environment steps: 2500000, seed: 96, eval reward: 1138.01
Average reward: 2683.79
```

Task 3 (3M):

```
Environment steps: 3000000, seed: 77, eval reward: 2280.25
Environment steps: 3000000, seed: 78, eval reward: 1522.09
Environment steps: 3000000, seed: 79, eval reward: 2494.37
Environment steps: 3000000, seed: 80, eval reward: 608.58
Environment steps: 3000000, seed: 81, eval reward: 1635.39
Environment steps: 3000000, seed: 82, eval reward: 2501.01
Environment steps: 3000000, seed: 83, eval reward: 4152.87
Environment steps: 3000000, seed: 84, eval reward: 2558.49
Environment steps: 3000000, seed: 85, eval reward: 511.03
Environment steps: 3000000, seed: 86, eval reward: 542.84
Environment steps: 3000000, seed: 87, eval reward: 1258.58
Environment steps: 3000000, seed: 88, eval reward: 3771.63
Environment steps: 3000000, seed: 89, eval reward: 4730.38
Environment steps: 3000000, seed: 90, eval reward: 1414.19
Environment steps: 3000000, seed: 91, eval reward: 2905.82
Environment steps: 3000000, seed: 92, eval reward: 485.35
Environment steps: 3000000, seed: 93, eval reward: 590.75
Environment steps: 3000000, seed: 94, eval reward: 4722.99
Environment steps: 3000000, seed: 95, eval reward: 2520.66
Environment steps: 3000000, seed: 96, eval reward: 3788.73
Average reward: 2249.80
```

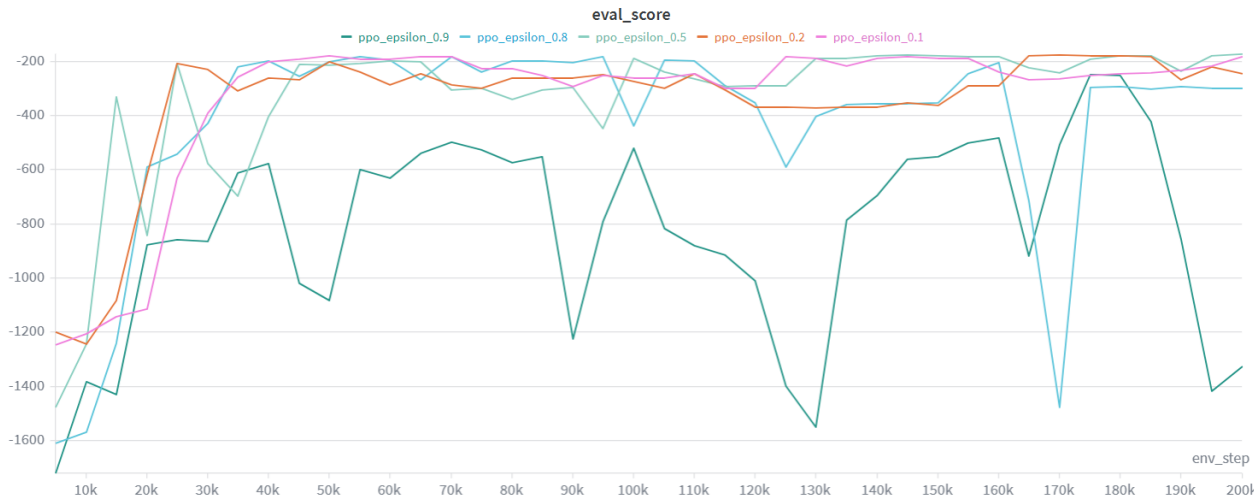
- Compare the sample efficiency and training stability of A2C and PPO (10%).

使用相同的種子評估時可以發現，A2C 的分數是逐漸上升，其中還伴隨著不少劇烈的波動，而 PPO 則在大約 30K env step 時就達到了高分，後面也一直非常穩定。由此可見，PPO 的樣本效率比 A2C 還高。

- Perform an empirical study on the key parameters, such as clipping parameter and entropy coefficient (5%).

Clipping parameter:

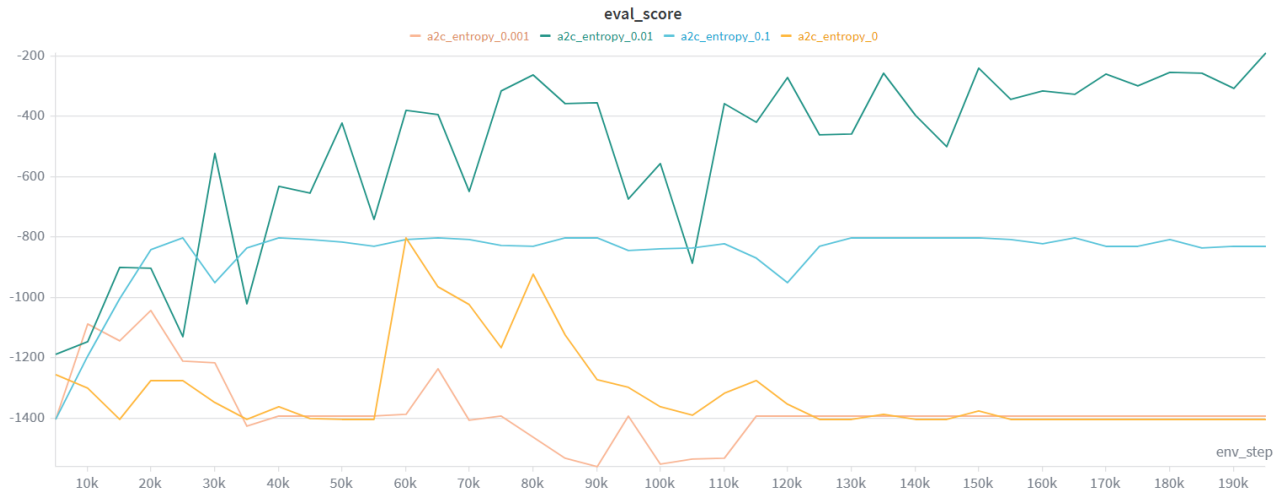
改變 epsilon，在 0.1, 0.2, 0.5 時，訓練結果都差不多，0.8 時開始不穩定，0.9 則變差很多。



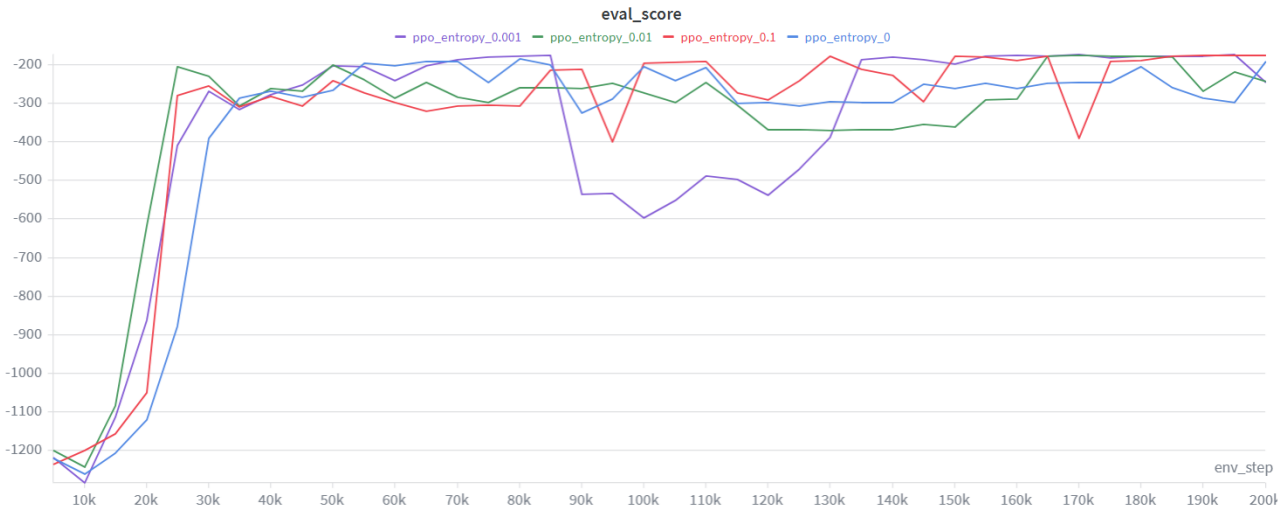
Entropy coefficient:

使用 A2C 演算法時，選擇的 entropy 很重要，0.01 表現最好，其次為 0.1，不是用 entropy 和極低的 entropy 作用差不多。然後當演算法換成 PPO 時，entropy 的大小影響就沒有那麼重要了，由圖中可以看出，四條曲線是非常接近，其中的差距比較像是隨機性造成的。

A2C:



PPO:



- Additional analysis on other training strategies (Bonus up to 10%)

分別進行四種測試

1. 移除 observation normalization：讓 state 不標準化，也就是維持原本的平均和標準差，不改成常態分佈

```
#env = gym.wrappers.NormalizeObservation(env)
#env = gym.wrappers.TransformObservation(env, lambda obs: np.clip(obs, -10, 10), observation_space=env.observation_space)
```

```
#norm_env = self.get_normalize_wrapper(self.env)
#obs_rms_state = None
#if norm_env is not None:
#    obs_rms_state = {
#        'mean': norm_env.obs_rms.mean,
#        'var': norm_env.obs_rms.var,
#        'count': norm_env.obs_rms.count
#    }
```

2. 移除 gradient clipping：使梯度沒有大小限制

```
#torch.nn.utils.clip_grad_norm_(self.critic.parameters(), 0.5)
```

```
#torch.nn.utils.clip_grad_norm_(self.actor.parameters(), 0.5)
```

3. 把 uniform 的 layer itit 換成 othogonal：讓特徵值絕對值改成絕對值為 1，初始時神經元學到的特徵都不同

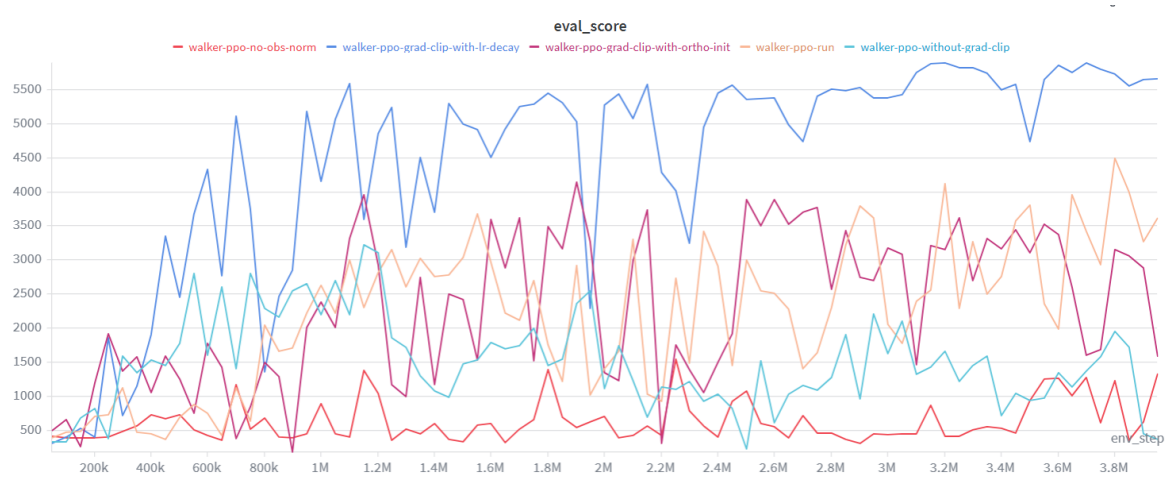
```
def init_layer_orthogonal(layer: nn.Linear, gain: float = 1.0) -> nn.Linear:
    """Init orthogonal parameters on the single layer with constant bias."""
    torch.nn.init.orthogonal_(layer.weight, gain=gain)
    torch.nn.init.constant_(layer.bias, 0.0)
    return layer
```

```
if use_ortho_init:
    init_layer_orthogonal(self.l1, gain=np.sqrt(2))
    init_layer_orthogonal(self.l2, gain=np.sqrt(2))
    init_layer_orthogonal(self.mu, gain=0.01)
    init_layer_orthogonal(self.std, gain=0.01)
```

4. learning rate decay：將 learning rate 改成隨著 episode 線性遞減，確保訓練後期不會更新過多

```
if self.use_lr_decay:
    lr_lambda = lambda step: max(0.0, 1.0 - (step / (self.num_episodes * self.rollout_len)))
    self.actor_scheduler = optim.lr_scheduler.LambdaLR(self.actor_optimizer, lr_lambda=lr_lambda)
    self.critic_scheduler = optim.lr_scheduler.LambdaLR(self.critic_optimizer, lr_lambda=lr_lambda)
```

訓練時 evaluation score：



結論：

observation normalization	gradient clipping	orthogonal layer init	learning rate decay
非常有用	有用	沒什麼用	非常有用

python a2c_pendulum.py --test --env-step 190000 --seed 700 --model-path ../LAB7_111550041_task1_a2c_pendulum.pt

python ppo_pendulum.py --test --env-step 30000 --seed 700 --model-path ../LAB7_111550041_task2_ppo_pendulum.pt

python ppo_walker.py --test --env-step 1000000 --seed 77 --model-path ../LAB7_111550041_task3_ppo_1m.pt

python ppo_walker.py --test --env-step 1500000 --seed 77 --model-path ../LAB7_111550041_task3_ppo_1p5m.pt

python ppo_walker.py --test --env-step 2000000 --seed 77 --model-path ../LAB7_111550041_task3_ppo_2m.pt

python ppo_walker.py --test --env-step 2500000 --seed 77 --model-path ../LAB7_111550041_task3_ppo_2p5m.pt

python ppo_walker.py --test --env-step 3000000 --seed 77 --model-path ../LAB7_111550041_task3_ppo_3m.pt

python ppo_walker.py --test --env-step -1 --seed 77 --model-path ../LAB7_111550041_task3_best.pt